

Project history and engineering report

Chronological account of the DSP console, processing chain, debugging work, real-time controls and audio test tools

REPORT DATE 1 October 2026

PROJECT SeaMic - browser-based Cmajor microphone DSP test console

PURPOSE

Explain how the project evolved, why important DSP and host decisions were made, how reported audio faults were investigated, and how to reproduce tests with microphone recordings or local WAV stimuli.

Scope and chronology note

This report reconstructs the project in the order recorded in the conversation handoff and verifies technical details against the current source and project notes. The original exchanges do not retain a reliable date for every milestone; the sequence is chronological, but dates are not invented. The report date is the only date applied to the complete account.

Executive summary

SeaMic grew from a static, dependency-free web sketch into a seven-stage Cmajor processing console. The work progressively clarified the signal architecture, separated modules into standalone patches, exposed bypass and Solo testing, added diagnostics, corrected numerical and envelope-following problems that could cause silence or clipped word endings, and documented which features are real versus still targets. This final delivery adds local recording and WAV stimulus playback through the same live DSP graph.

01 | Starting brief: a lightweight SeaMic web workspace

The project began with a deliberately small frontend brief: create a visual home for the SeaMic microphone DSP pipeline using only `index.html`, `styles.css` and `app.js`, plus project instructions and short architecture notes. The requested stack was plain HTML/CSS/JavaScript, no frameworks, dependencies, build, fetch, or ES modules, with Spanish-facing copy and no Cmajor integration at that stage. The first objective was orientation: make the signal flow understandable before

expanding the DSP implementation.

- The first visual was a simple introduction and a card diagram for the stages already known: DC block, K-weighting, VAD, AGC, Soft Gate and Limiter.
- The workspace structure also reserved legacy and clean module areas, documentation, editor settings and Copilot guidance so later imports had a clear home.

02 | Moving from a diagram to a working Cmajor handoff

The next project phase connected the web workspace to SeaMic Cmajor and prepared the console to host the DSP UI. Modules were imported from a separate workspace, so the handoff asked for code and naming cleanup rather than a blind copy. The console was changed to a vertical, block-based rack: engine/session controls first, then sequential plugins, then output. This established a physical-console mental model and gave each stage a clear place for its controls and status.

- The initial expanded chain was Input -> AEC -> Loudness -> Output. The module sequence was explicitly expected to grow by inserting plugin stages between input and output.
- The architecture footer/diagram was made to show ADC -> DSP1 -> DSP2 ... -> DA, and then relabelled with meaningful DSP names rather than leaving numbered generic blocks.

03 | Defining the intended broadcast voice architecture

The chain evolved from a small set of filters into a planned broadcast voice path. The placement decisions mattered: AEC uses far-end playback as its reference; De-Reverb comes immediately after AEC so room tails do not hold the voice detector open; loudness normalization precedes Tone so the nonlinear stage sees a more consistent level; and the limiter is last to contain peaks generated by earlier processing.

- The target order became ADC (16 kHz, float) -> DC Offset Removal -> AEC -> De-Reverb -> Noise Gate/VAD -> Loudness Normalisation -> Tone/Warmth -> Look-Ahead Limiter -> Output -> Opus -> Network.
- The architecture was split into independent Cmajor patches where needed, rather than presenting a diagram with missing functional stages. Transport stages Opus and Network remain labelled targets; they are not implemented in these patches.

04 | Console identity, observability and module testing

A later design brief defined a strict retro desktop console: hard borders and bevels, Tahoma-like UI type, monospace numeric readouts, scientific data colors, and no soft gradients, glass effects, blurred shadows or chart decoration. This kept the interface consistent with an AnyDesk One / Windows desktop visual language while reserving color for signal categories and alerts.

- The interface remained plain HTML, CSS and JavaScript with no third-party runtime libraries. The generated Cmajor WebAudio runtimes are the explicit exception required to run the DSP; they are generated by the Cmajor tool, not manually authored libraries.
- Every module starts bypassed. Bypass is described as a dry pass-through state, and controls say Activate or Bypass so the current state and the next action are not conflated.
- Per-module Solo isolates one processor for fault-finding, bypassing the others. Module detail disclosures were added to explain implementation and show representative Cmajor source, rather than presenting aspirational math as if it were running code.

05 | Seven standalone stages and application playback reference

The web host now loads seven independent patches in series: SeaMicDCOffset, SeaMicAEC, SeaMicDeReverb, SeaMicNoiseGateVAD, SeaMicLoudnessNormalization, SeaMicToneWarmth and SeaMicLookAheadLimiter. A composite graph remains for command-line work but is not confused with a browser module. The AEC stage has a second, distinct signal path: the SeaTime playback node is routed to the speakers and tapped before output as the far-end reference.

- AEC uses an adaptive NLMS filter with adjustable filter length and step size, and can freeze adaptation under double-talk. It stays locked/bypassed without a live playback reference; the console does not perform system loopback capture.
- DC Offset removal filters both microphone and reference paths. The noise detector also supplies a speech flag to the loudness AGC sidechain; Tone includes a post-shaper DC blocker.
- Generated bundles live under web/generated/<patch>/ and must be regenerated from Cmajor source. They are not hand-edited. Cmajor WebAudio requires the local HTTP server; file:// is not supported for these module bundles.

06 | Audio failures: loudness/tone silence and numerical stability

A reported failure was that enabling modules 5 and 6 (Loudness and Tone) could stop speaker output. The project notes identify a root cause in the AGC rolling RMS sum: floating-point subtraction can leave a tiny negative value even though the mathematical sum of squares cannot be negative. Taking sqrt of that negative residue creates NaN; the non-finite gain then contaminates AGC output and the downstream Tone stage, yielding silence. The AGC accumulator is now clamped to zero before RMS calculation. The VAD rolling RMS accumulator received the same numerical guard.

- This fix addresses a numeric invalid-state path; it is not an audio level threshold or a substitute for listening tests.
- Synthetic tests were used to verify the dry/DSP routes and Loudness/Tone recovery after silence. No real microphone was opened during the validation described in the handoff.

07 | De-Reverb artifacts and clipped word endings

The De-Reverb processor is an envelope shaper, not blind deconvolution or a complete acoustic de-reverberator. Its earlier settings allowed materially faster gain modulation and a wider gain span (roughly -12 dB attenuation to +3 dB boost, with short symmetric smoothing). Fast envelope-driven gain changes can sound like pumping/glitches and can reduce the level that reaches the voice detector. The code was made conservative: no boost, a floor near -6 dB, and slower gain-reduction/recovery smoothing of 20/120 ms.

- The VAD is a 32 ms rolling RMS and falling noise-floor/SNR hysteresis detector, not the proposed Otsu classifier. Its enter/exit thresholds are 9/4 dB SNR.
- Word endings could be classified as inactive after the short original hangover; a second forced-close timer could override that hold, and the gate could fade down abruptly. The forced-close path was removed, VAD hangover increased to 900 ms, gate hold remains an additional 400 ms and attenuation is ramped over 30 ms.
- A 0.001 RMS absolute quiet close guard (~-60 dBFS) prevents speech from staying latched when the learned floor becomes extremely low. This fixed threshold is a known tuning risk and needs representative microphone/noise measurements before changing.

08 | Making code explanations match the processors

The seven module disclosures were rewritten from mathematical descriptions to implementation descriptions. The purpose was to make the console a debugging aid: describe the actual running equations/controls in everyday language and show source that corresponds to the implementation, while explicitly marking roadmap differences such as Otsu VAD, a symmetric Tone curve/shelf, integrated EBU R128, and true-peak limiting as pending rather than claiming they already exist.

- Documentation was synchronized across the console, pipeline notes, Cmajor module notes, microphone test guide, VAD debug notes and project memory. Historical test results are labelled historical where processor behavior has since changed.

09 | Exposing parameters that actually exist in Cmajor

The generic Cmajor patch view appeared empty in the browser, so the console was changed to render its own ordinary HTML controls and send each change directly to the patch endpoint with `sendEventOrValue`. Ranges and initial values are taken from Cmajor endpoint annotations. Controls stay visible before start and become usable once patches have loaded; values are also visible numerically for reproducible tuning.

- AEC: filter length 32-1024 taps (512 default), adaptation step size 0.01-2 (0.5 default), double-talk detector on/off.
- De-Reverb: amount 0-1 (0.5 default). Loudness: target -30 to -6 dBFS (-19), maximum gain 0-40 dB (30), minimum gain -24 to 0 dB (-12).
- Tone: drive 1-8 (1.5), harmonic asymmetry 0-0.5 (0.2), output gain -12 to +6 dB (-3). DC cutoff, VAD/gate timing and thresholds, and limiter ceiling/look-ahead/release are currently fixed and are stated as such, not represented by fake sliders.

10 | Current delivery: record and replay diagnostic stimuli

The newest request adds repeatable test input without changing the Cmajor chain. The audio-test block is placed beside engine controls. Recording captures the microphone locally as mono 16-bit PCM WAV at the active AudioContext rate, with a five-minute cap; the resulting buffer can be replayed through the same input merger and all seven serial patches or downloaded as a WAV file.

- A user can select a local WAV, inspect its duration/channel/rate, and play it through that same chain. WebAudio decodes/resamples it to the active context; the mono processing input receives the browser downmix for multichannel material.
- During test playback the live microphone path is temporarily muted to prevent test signal mixing. The test source feeds the normal input monitor and the DSP chain, so it can be heard dry, through individual active stages/Solo, or through the full chain. The monitor route label changes to TEST AUDIO while it is active.
- The application playback/AEC reference bus is separate and unchanged; playing a test file as microphone input does not fabricate an AEC reference. All capture and file decoding stay in the browser; no upload or server-side audio processing was introduced.

11 | Verification, usage and remaining boundaries

Editor diagnostics reported no HTML, CSS or JavaScript issues. Browser smoke tests used an oscillator-backed synthetic microphone and a generated 440 Hz WAV: all seven Cmajor modules

loaded; microphone capture produced a 16 kHz WAV buffer; the recording was replayable and downloadable; the local WAV decoded and played; an active Tone stage showed TEST AUDIO -> seven DSP modules -> output and a measurable output level; Stop returned the host to 0/7 loaded with no browser console errors. A prior Cmajor validation compiled all seven standalone patches and passed three AEC tests.

- To test locally, start the included static server, open <http://127.0.0.1:8001/>, start the engine, grant microphone permission if recording, and either record a short sample or load a WAV. Keep the chain bypassed for the dry reference, then activate one module at a time, use Solo, or enable the full chain. Use headphones for microphone monitoring.
- No real microphone or SeaTime playback path was exercised in the latest browser tests. Real AEC convergence, real-room De-Reverb behavior, VAD tail behavior across microphones, and subjective sound quality remain to be measured by the user.
- The browser reports its negotiated sample rate; requesting 16 kHz does not guarantee every browser/device will honor it. The Cmajor bundles still require HTTP. Opus/network transport and the target algorithms explicitly marked pending remain outside this delivery.
- Recording currently uses the broadly supported but deprecated WebAudio ScriptProcessorNode to obtain PCM samples for WAV construction; current Firefox/Chromium smoke testing succeeded. Replacing it with a dedicated AudioWorklet recorder is a reasonable future hardening step if the target browser deprecates/removes ScriptProcessorNode.

12 | Source map and project artifacts

Primary application files are `web/index.html`, `web/styles.css` and `web/app.js`. Standalone patch graphs and metadata are in `cmajor/patches/`. DSP processors and AEC implementation are in `cmajor/modules/dsp/` and `cmajor/modules/aec/` with legacy chain processors at the project root. Architecture and test constraints are described in `docs/pipeline-notes.md`, `web/README.md`, `MIC_TEST.md`, `VAD_DEBUG.md` and `MEMORY.md`. This report is the chronological overview; those files remain the detailed implementation references.